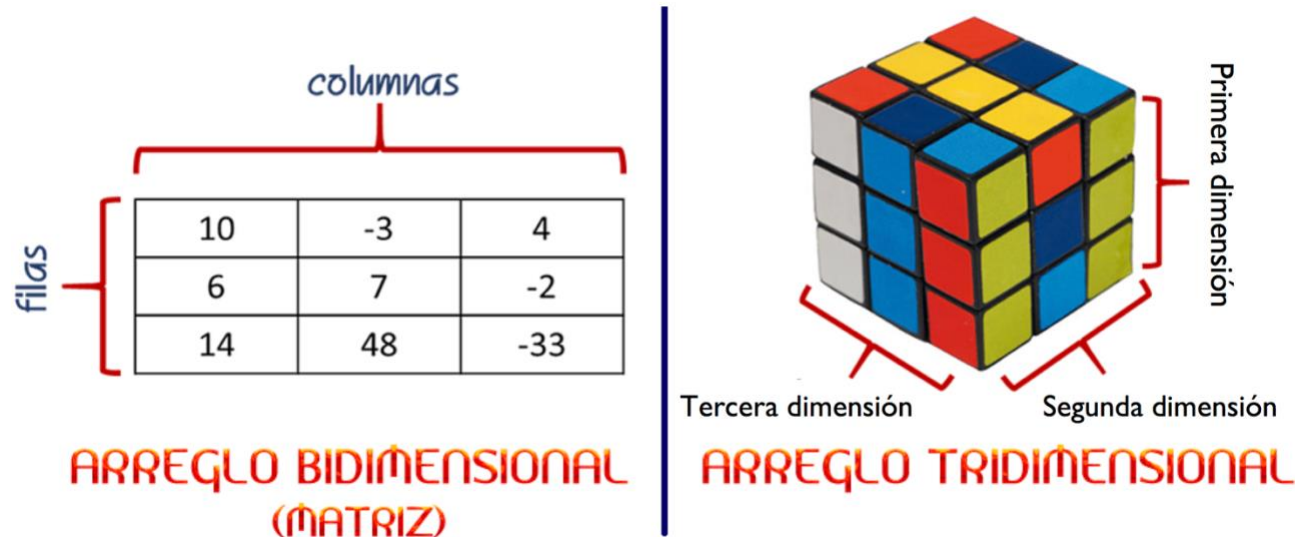


PRELIMINARES

Introducción

Aunque las listas puedan ayudarnos a resolver varios problemas, el que sean de una sola dimensión reduce su aplicabilidad. Para resolver este problema existen listas/arreglo multidimensionales.



Una matriz (lista bidimensional) tiene un número determinado de filas y columnas.

Su creación y manejo es similar a la creación unidimensional, pero con dos índices. Para acceder al dato i, j de la matriz llamada MATRIZ, escribimos $MATRIZ[i][j]$, donde i hace referencia a la fila de la matriz y j hace referencia a la columna. Tal como en el caso de una dimensión, al acceder a este elemento es posible obtener su valor o modificarlo.

Ejemplo:

```
arreglo1 = [[0,0],[0,0]]
arreglo1[0][0] = 10
print(arreglo1) #muestra por pantalla [[10, 0], [0 0]]
print(arreglo1[0][1]) #muestra por pantalla 0
```

Para obtener el número de filas de la matriz podemos utilizar la función `len()`. Sin embargo, el número de columnas de cada fila tiene que obtenido en forma particular para cada fila.

Ejemplo:

```
arreglo1 = [[0,0],[0,0],[0,0]]
print(len(arreglo1)) #muestra por pantalla 3
```

```
print(len(arreglo1[1])) #muestra por pantalla 2, el número de columnas de la 2da fila
```

Como la lista bi-dimensional es una lista de listas, el operador `:` permite obtener acceso a múltiples filas de la matriz (pero no a múltiples columnas). En forma particular el operador `arreglo[i1:i2]` retorna una lista doble con las filas correspondiente a los índices `i1` a `i2-1`.

Ejemplo, calcular el promedio simple de 40 alumnos donde cada alumno tienen 3 notas generadas en forma aleatoria

```
import random

#Generación de notas aleatorias
notas = []
for i in range(0,40,1):
    notasEstudiante=[]
    for j in range(0,3,1):
        notasEstudiante.append(round(random.random()*6+1,1))
    notas.append(notasEstudiante)

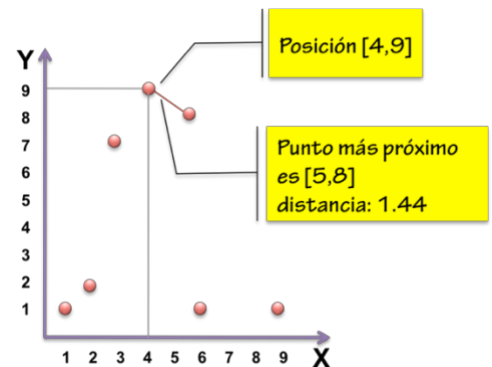
#Generación de promedios
promedios=[]
for i in range(0,40,1):
    notasEstudiante=notas[i]
    promedios.append(round(sum(notasEstudiante)/len(notasEstudiante),1))
print(promedios)
```

Como se puede observar, al obtener una sub lista más pequeña podemos utilizar las funciones previamente aprendidas, tales como `sum()` y `len()`. Recuerde, siempre es bueno simplificar el problema antes de resolverlo, por eso al calcular el promedio de cada estudiante, se obtuvo la lista con las notas de ese estudiante en particular. El resto de las funciones también pueden ser aplicadas de esta misma manera.

PROBLEMAS PROPUESTOS

- Suponga que tiene una lista de dos dimensiones de igual tamaño de largo 7 que representan puntos en el espacio cartesiano X-Y. Realice un programa que por cada coordenada determine la coordenada más cercana. Indique dicha distancia. (👉: utilice ciclos dobles). Ocupe los siguientes datos de entrada:

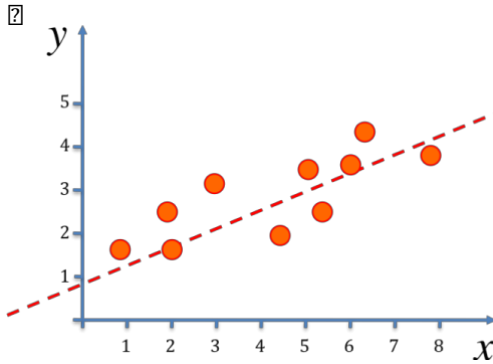
```
puntos=[[1,1], [2,2], [3,7], [4,9], [5,8], [6,1], [9,1]]
```



```
import math
puntos=[[1,1], [2,2], [3,7], [4,9], [5,8], [6,1], [9,1]]
for i in range(0,len(puntos),1):
    #punto al que se le calculará la distancia a sus vecinos
    puntoBase=i

    #Calculando distancia
    distanciaMinima=math.inf
    indexMinimo=-1
    for j in range(0,len(puntos),1):
        if i!=j:
            distanciaTemporal=math.sqrt(((puntos[puntoBase][0]-
puntos[j][0])**2+(puntos[puntoBase][1]-puntos[j][1])**2))
            if distanciaTemporal<distanciaMinima:
                distanciaMinima=distanciaTemporal
                indexMinimo=j
    print("El punto más cercano del punto ",puntos[puntoBase]," es
",puntos[indexMinimo]," a una distancia de ",distanciaMinima)
```

2. Usted dispone de un conjunto de datos en una matriz de 10 x 2, donde la primera columna corresponde a la posición del eje x y la segunda columna es la posición del eje y . Realice un programa que ajuste dichos datos a una ecuación de la recta, es decir, encuentre los parámetros m , b de la ecuación de la recta $y = m \cdot x + b$.



	x	y
1	0.9	1.7
2	2.1	1.5
3	1.9	2.5
4	3	3.1
5	4.5	1.8
6	5	3.5
7	5.4	2.2
8	6	3.3
9	6.2	4.5
10	7.9	3.9

Para resolver este problema, de las estadísticas sabemos que m y b se resuelven de la siguiente forma:

$$m = \frac{n \cdot S_{XY} - S_X S_Y}{n \cdot S_{XX} - S_X^2}$$

$$b = \frac{S_{XX} S_Y - S_X S_{XY}}{n S_{XX} - S_X^2}$$

donde:

$$S_X = \sum_{i=1}^n x_i \quad S_Y = \sum_{i=1}^n y_i \quad S_{XY} = \sum_{i=1}^n x_i \cdot y_i \quad S_{XX} = \sum_{i=1}^n x_i^2$$

El símbolo $\sum$ significa sumatoria, es decir, la suma de valores parciales, y n es el número de datos del problema.

```
puntos=[[0.9,1.7],[2.1,1.5],[1.9,2.5],[3,3.1],[4.5,1.8],[5,3.5],
[5.4,2.2],[6,3.3],[6.2,4.5],[7.9,3.9]]

n=len(puntos)
Sx=0
Sy=0
Sxy=0
Sxx=0
for i in range(0,n,1):
    Sx=Sx+puntos[i][0]
    Sy=Sy+puntos[i][1]
    Sxy=Sxy+puntos[i][0]*puntos[i][1]
    Sxx=Sxx+puntos[i][0]*puntos[i][0]
m=round((n*Sxy-Sx*Sy)/(n*Sxx-Sx*Sx),2)
b=round((Sxx*Sy-Sx*Sxy)/(n*Sxx-Sx*Sx),2)
print("La ecuación de la recta es X=",b,"+",m,"*X")
```

3. El invierno se acerca rápidamente y usted muy precavido ha decidido instalar un nuevo sistema de calefacción en su hogar. Para ello usted cotiza distintos modelos de calefacción, desde estufas a parafina, sistemas eléctricos, calefacción a leña, termopanel, etc. (Tabla 1)

Tabla 1. Cotizaciones de calefacción de 10 sistemas

Sistema	Costo mensual (c)	Calor (B)	Área (A)
1	\$51.000	5000	35
2	\$37.500	3500	25
3	\$42.700	3400	30
4	\$67.000	5100	38
5	\$15.000	2100	15
6	\$35.000	3600	28
7	\$28.000	3250	25
8	\$17.000	2300	12
9	\$43.000	3700	32
10	\$29.000	3100	30

Cada uno de estos dispositivos tiene un costo de mantención diario (d), un rendimiento de calor por hora (B) y un rendimiento por área cuadrada (A). Dado sus conocimientos en programación, diseñe un programa que busque el mejor sistema tal que minimice la ecuación de rendimiento y maximice el área cubierta por el sistema.

$$\text{rendimiento} = \frac{d \cdot A}{B},$$

donde d es el costo diario, c es el costo mensual ($c = d \cdot 30$). Su programa debe emplear los datos de la tabla 1 almacenados en una matriz. Al finalizar, su programa debe indicar cuál es el mejor sistema.

```
Calefaccion=[[51000,5000,35],[37500,3500,25],[42700,3400,30],[67000,5100,38],
,[15000,2100,15],[35000,3600,28],[28000,3250,25],[17000,2300,12],[43000,3700,32],
,[29000,3100,30]]

n=len(Calefaccion)
rendimiento=[]
for i in range(0,n,1):
    rendimiento.append(Calefaccion[i][0]*Calefaccion[i][2]/Calefaccion[i][1])

minRendIndex=0
for i in range(0,n,1):
    if rendimiento[i]<rendimiento[minRendIndex]:
        minRendIndex=i

print("El mejor sistema es el número ",minRendIndex)
```

4. Pídale al usuario que ingrese un número entero "n". Cree una lista de dos dimensiones de tamaño "n" utilizando ciclos anidados. Muestra la matriz de identidad resultante. La matriz identidad es una matriz de 0 donde la diagonal principal tiene valores 1.

```
n=int(input("Ingrese numero de filas: "))
matriz=[]
for i in range(1,n+1):
    fila=[]
    for j in range(1,n+1):
        if i==j:
```

```
        fila.append(1)
    else:
        fila.append(0)
    matriz.append(fila)

#Mostrando matriz
for i in range(0, len(matriz)):
    print(matriz[i])
```

5. Usando su conocimiento de listas cree un programa que solicite al usuario un número N y un número M, posteriormente usted deberá crear una matriz de NxN con números aleatorios entre 1 y M. Después de esto su código debe hacer las siguientes acciones:
- Mostrar la matriz creada
 - Solicitar al usuario un tercer número que debe estar entre 1 y M
 - Recorrer la matriz y cambiar a 0 todos los números que se encuentren alrededor del número seleccionado
 - Mostrar por pantalla la nueva matriz

```
import random
N=int(input("Ingrese numero de filas y columnas: "))
M=int(input("Ingrese máximo número a generar: "))
matriz=[]
for i in range(0,N,1):
    fila=[]
    for j in range(0,N,1):
        fila.append(random.randint(1,M))
    matriz.append(fila)

#Mostrando la matriz
for i in range(0, len(matriz)):
    print(matriz[i])

#Pidiendo el numero
numUsuario=0
while (numUsuario<1 or numUsuario>M):
    print("Ingrese un numero entre 1 y ",M)
    numUsuario=int(input())

#Borrando valores alrededor de 0
for i in range(0,N,1):
    for j in range(0,N,1):
        if matriz[i][j]==numUsuario:
            #Se activa el proceso de borrar vecinos
```

```

for k in range(-1,2,1):
    for l in range(-1,2,1):
        newIndexI=i+k
        newIndexJ=j+l
        #Verificando el indice I este dentro de la matriz
        if newIndexI>=0 and newIndexI<N:
            #Verificando el indice J este dentro de la matriz
            if newIndexJ>=0 and newIndexJ<N:
                #Verificando que no borre el número seleccionado:
                if matriz[newIndexI][newIndexJ]!=numUsuario:
                    matriz[newIndexI][newIndexJ]=0

#Mostrando la matriz
for i in range(0,len(matriz)):
    print(matriz[i])

```

6. Este es el juego del gato y el ratón. Usted dispone de una malla cuadrada de 5×5 y comienza en la posición (1,1). El ratón se encuentra en una posición aleatoria de la malla (definida con la función `random.randint`). A medida que usted se mueve en el tablero, el ratón puede moverse a cualquier lugar del tablero, ya que es un ratón saltador. Para que usted pueda moverse en el tablero, el programa le debe preguntar cada vez, si se mueve a la izquierda, derecha, arriba o abajo. Si el ratón, y usted se encuentra en la misma posición del tablero, usted gana el juego y el programa termina.

	1	2	3	4	5
1	J				
2					
3			R		
4					
5					

```

import random
n=5
matriz=[]
for i in range(1,n+1):
    fila=[]
    for j in range(1,n+1):
        fila.append(0)
    matriz.append(fila)

#Representado al jugador
jugadorI=0
jugadorJ=0
matriz[jugadorI][jugadorJ]="J"

#Representado al raton
ratonI=random.randint(0,4)
ratonJ=random.randint(0,4)
matriz[ratonI][ratonJ]="R"

```

```
#Impresion mejor
for i in range(0,len(matriz)):
    print(matriz[i])

if (ratonI==jugadorI) and (ratonJ==jugadorJ):
    bandera=False
else:
    bandera=True

while(bandera==True):
    #Mover a la persona
    movimiento=int(input("hacia donde te mueves, 0: abajo, 1: arriba, 2: izq,
3: der"))

    banderaMovimiento=True
    #Verificar si el movimiento es valido
    if movimiento==0 and jugadorI==4:
        banderaMovimiento=False
    if movimiento==1 and jugadorI==0:
        banderaMovimiento=False
    if movimiento==2 and jugadorJ==0:
        banderaMovimiento=False
    if movimiento==3 and jugadorJ==4:
        banderaMovimiento=False

    #Moviendo al jugador
    if banderaMovimiento==True:
        matriz[jugadorI][jugadorJ]=0
        if movimiento==0:
            jugadorI=jugadorI+1
        if movimiento==1:
            jugadorI=jugadorI-1
        if movimiento==2:
            jugadorJ=jugadorJ-1
        if movimiento==3:
            jugadorJ=jugadorJ+1
        matriz[jugadorI][jugadorJ]="J"

    #Moviendo al raton
    matriz[ratonI][ratonJ]=0
    ratonI=random.randint(0,4)
    ratonJ=random.randint(0,4)
    matriz[ratonI][ratonJ]="R"
```



```
for i in range(0,len(matriz)):
    print(matriz[i])

if (ratonI==jugadorI) and (ratonJ==jugadorJ):
    bandera=False
else:
    bandera=True

else:
    print("Movimiento invalido")

print("Felicitaciones pillaste al raton")
```

7. Dada la matriz sudoku (matriz de 9x9), verifique que la matriz cumpla con los requerimientos de un Sudoku. Esto implica que cada fila tenga valores entre 1 y 9 (sin que se repita ningún número), que cada columna tenga valores entre 1 y 9 (sin que se repita ningún número), y que cada sub matriz de 3x3 tenga valores entre 1 y 9 (sin que se repita ningún número). Si esto se cumple, deberá imprimir por pantalla “Sudoku correcto”, caso contrario, deberá imprimir “Sudoku incorrecto”.
Hint: puede utilizar un arreglo de tamaño 9 para verificar si un número ha salido.

```
sudoku = [
    [5, 3, 4, 6, 7, 8, 9, 1, 2],
    [6, 7, 2, 1, 9, 5, 3, 4, 8],
    [1, 9, 8, 3, 4, 2, 5, 6, 7],
    [8, 5, 9, 7, 6, 1, 4, 2, 3],
    [4, 2, 6, 8, 5, 3, 7, 9, 1],
    [7, 1, 3, 9, 2, 4, 8, 5, 6],
    [9, 6, 1, 5, 3, 7, 2, 8, 4],
    [2, 8, 7, 4, 1, 9, 6, 3, 5],
    [3, 4, 5, 2, 8, 6, 1, 7, 9]
]

#Asumo que el sudoku es correcto y se cambia a False en caso que no
bandera=True
#Verificando filas
for i in range(0,len(sudoku)):
    #lista para ver si sale cada numero en la fila
    verif=[0,0,0,0,0,0,0,0,0]
    for j in range(0,len(sudoku)):
        verif[sudoku[i][j]-1]=1
    if sum(verif)!=9:
```

```
bandera=False

#Verificando columnas
for i in range(0,len(sudoku)):
    #lista para ver si sale cada numero en la fila
    verif=[0,0,0,0,0,0,0,0,0]
    for j in range(0,len(sudoku)):
        verif[sudoku[j][i]-1]=1
    if sum(verif)!=9:
        bandera=False

#Verificando cada subcuadrante de 3x3
for i in range(0,len(sudoku),3):
    for j in range(0,len(sudoku),3):
        verif=[0,0,0,0,0,0,0,0,0]
        for itI in range(i,i+3):
            for itJ in range(j,j+3):
                verif[sudoku[itI][itJ]-1]=1
        if sum(verif)!=9:
            bandera=False

print(bandera)
```